

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики
	Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка информации	Структурированные файлы Парсинг данных Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Тема 2

Беляков Андрей Юрьевич

к.т.н., доцент

gitverse.ru/band/TOI

Раздел 1. Информация и данные

Тема 1

Основные понятия информатики

- данные **vs** информация
- что изучает информатика?
- свойства информации?
- как измерить информацию?
- бит, байт, килобайт...
- аналоговые и цифровые сигналы

Тема 2

Кодирование информации

- целые числа
- вещественные числа
- кодирование символов
- таблицы символов
- Строки

gitverse.ru/band/TOI

Тема 2

Кодирование информации



Вопрос 1: Кодирование текста

- ASCII, Unicode, строки

Вопрос 2: Кодирование чисел

- целые, вещественные

gitverse.ru/band/TOI

Обсудим понятия

шифрование, хеширование, кодирование

Кодирование - преобразование информации из одной формы в другую

- из формы понятной человеку в формат,
подходящий для обработки компьютером

Декодирование - обратный процесс

- если известен стандарт

Компьютеры:

- десятичный
- троичный
- двоичный

Шифрование - преобразование данных с целью скрыть содержание

- обратный процесс (при наличии ключа)

Хеширование - преобразование данных с целью скрыть содержание

- НЕТ обратного процесса

Кодирование

ВИДЫ

преобразование исходной информации в формат, подходящий для обработки компьютером

По содержанию:

- символов (чисел, текста),
- медиа (изображений, звука, видео)

По способу представления:

- равномерное (ASCII, Unicode, Бэкона, Бодо)
- неравномерное (Фано, Морзе,)

Вопрос 1

Кодирование текста

ключевая идея:

- каждому символу сопоставляется уникальный числовой код

`gitverse.ru/band/TOI`

Кодирование СИМВОЛОВ

равномерное и неравномерное

Равномерное

Бэкона

Бодо

ASCII, Unicode

каждый символ кодируется последовательностью одинаковой длины для простоты кодирования, записи и распознавания

символы кодируются исходя из частоты их появления в тексте так, чтобы наиболее частый символ кодировался наиболее короткой последовательностью

Неравномерное

Фано

Морзе

Кодирование СИМВОЛОВ

равномерное и неравномерное

Равномерное

Бэкона

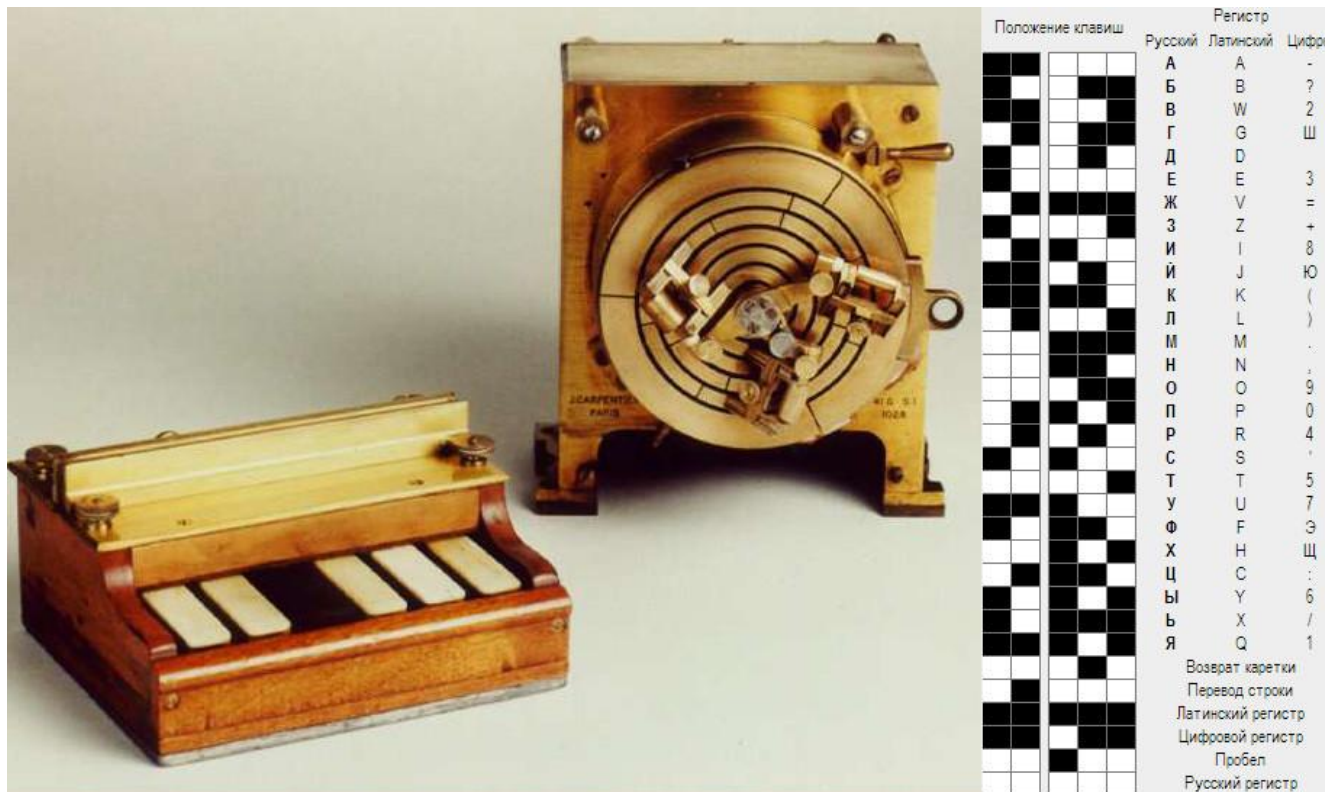
Бэкон в 1605 году предложил двужначную систему кодирования текста равномерным кодом для тайнописи, передачи сообщений с помощью вспышек света или звона колокола.

A	B	C	D	E	F
aaaaa	aaaab	aaaba	aaabb	aabaa	aabab
G	H	I, J	K	L	M
aabba	aabbb	abaaa	abaab	ababa	ababb
N	O	P	Q	R	S
abbaa	abbab	abbba	abbbb	baaaa	baaab
T	U, V	W	X	Y	Z
baaba	baabb	babaa	babab	babba	babbb

буквы I,J и U,V не различались во времена Френсиса Бэкона

Кодирование СИМВОЛОВ

равномерное и неравномерное



Равномерное - система Бодо

Самым частым буквам соответствуют комбинации с меньшим нажатием - меньше износ оборудования и нагрузка на пальцы

Буквы алфавитов сопоставляются по фонетическому принципу

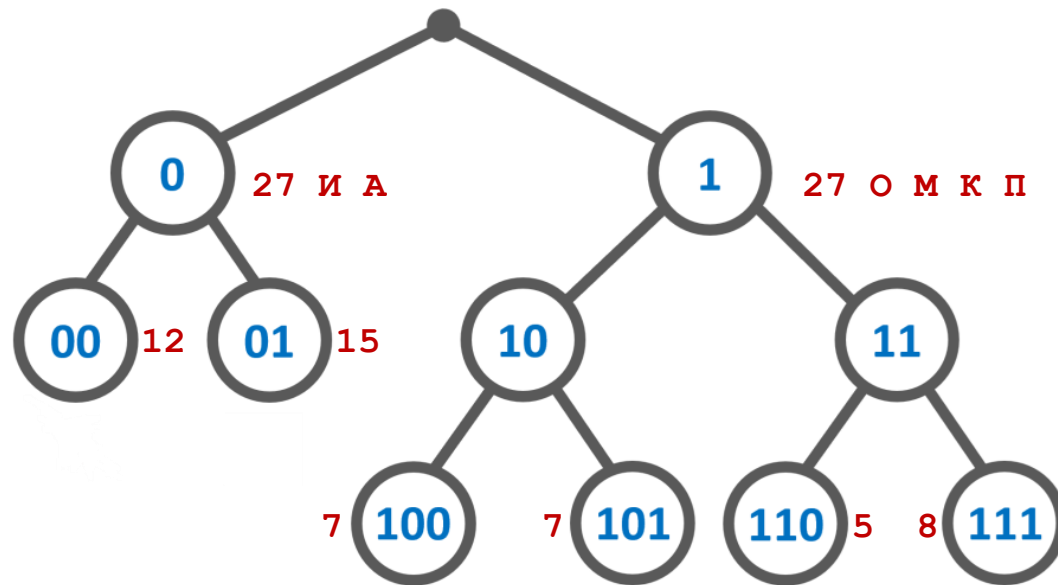
- этот принцип с 1856 года в русском варианте азбуки Морзе и впоследствии принят в кодировках КОИ-7 и КОИ-8

Французский инженер Бодо, 1872, телеграфный аппарат с клавиатурой из 5 клавиш: 2 для левой и 3 для правой руки, **1 символ = 5 битный двоичный код** (клавишами). На основе **кода Бодо** в 1924 году был принят международный телеграфный код.

Кодирование СИМВОЛОВ

неравномерное – условие Фано

Условие Фано: никакое кодовое слово не должно быть началом другого кодового слова



[П, К, М, О, И, А]

[5, 7, 7, 8, 12, 15]

частотность букв в тысячах

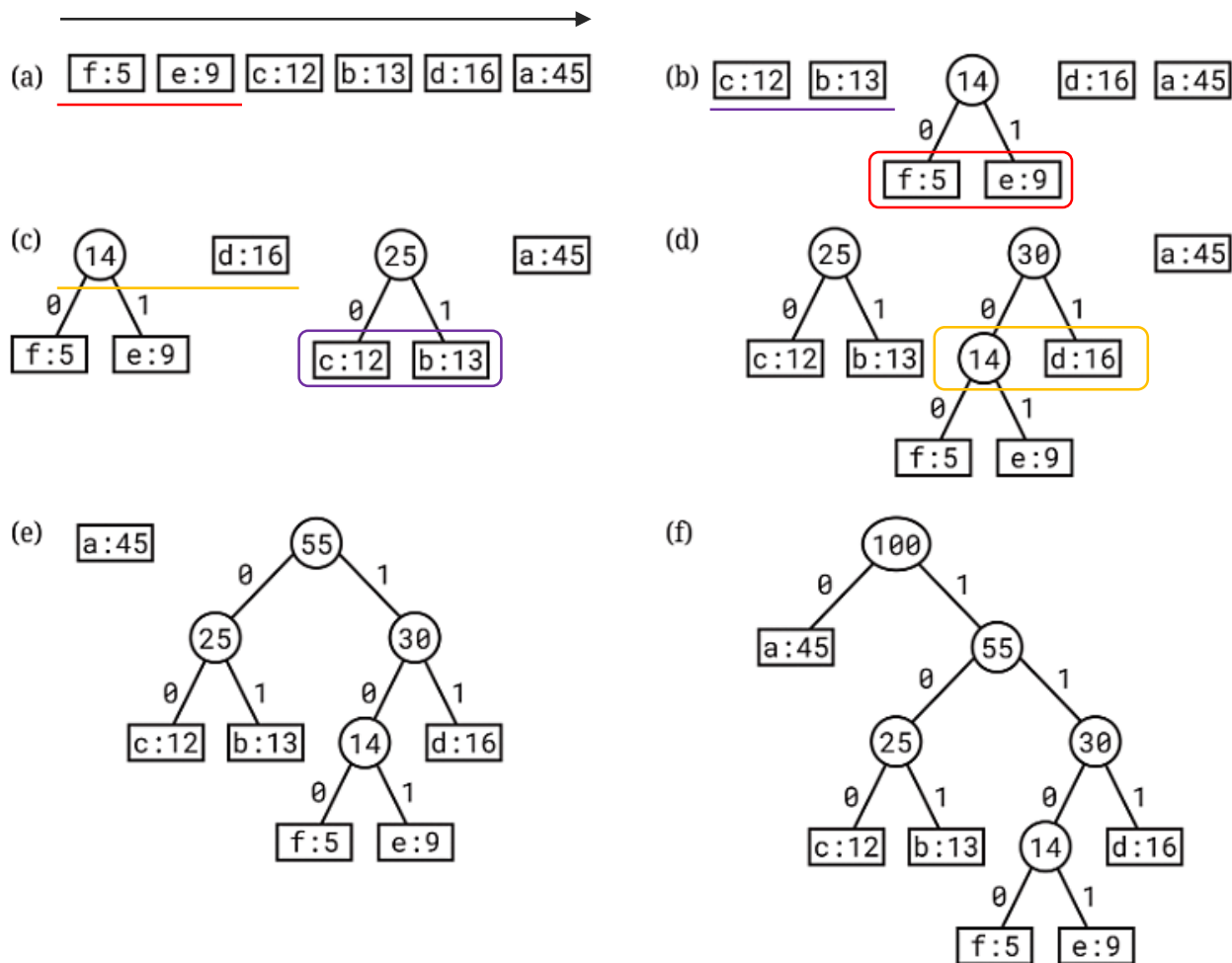
Порядок построения кода Фано:

1. Расположить все символы в порядке убывания частоты
2. Разделить их на две группы так, чтобы суммарные частоты в группах были примерно равны
3. Первой группе присвоить 0, второй - 1.
(добавить к текущей кодовой записи)
4. Продолжать делить каждую группу, пока каждый символ не получит уникальный код

- разным символам соответствуют кодовые слова разной длины
- цель - сделать меньше длину передаваемого сообщения за счёт того, что самые частые буквы сделать самыми короткими

Кодирование СИМВОЛОВ

неравномерное – алгоритм Хаффмана



Роберт Фано (1917-2016)
работал с Клодом Шенноном,
сформулировал условие Фано
его ученик Дэвид Хаффман
разработал алгоритм сопоставления символов
и их кодов на основе префиксного дерева

Кодирование символов – равномерное

начало формирования стандарта

ASCII

American Standard Code for Information Interchange

7 бит => 128 СИМВОЛОВ

0–31 – управляющие

32–126 – печатные

127 – DEL (удаление)

Кодировка - правила перевода символа в число и обратно, задаётся таблично

- символ DEL был принят для бумажных перфолент чтобы "стереть" ошибочно пробитый символ пробивали все отверстия (все 1) в этой позиции получившийся код (все биты = 1) стал символом DEL

Проверить программой символы, апробировать методы ord и chr

Продолжение формирования стандарта

ASCII

8 бит => 256 СИМВОЛОВ

0–127 – символы таблицы 7 бит

128–255 – национальные кодировки

Проблема в путанице при открытии

CP1251 – кириллица

CP1252 – западноевропейские алфавиты

возможны «кракозябры»

Проверить в
Notepad++

Продолжение формирования стандарта

UNICODE

первые 128 байт совпадают с ASCII

постоянно развивается – сейчас там более 150_000 символов
эмодзи, иероглифы, европейские языки, кириллица, древние, служебные

популярны UTF-8, UTF-16, UTF-32

UTF-8 – переменной длины от 1 до 4-х байт

UTF-8

старший бит всегда 0	Диапазон символов		Длина (байт)	Пример
	U+0000 ... U+007F	0 ... 127	1 байт	'A' → 0x41
	ASCII – английский алфавит, цифры			
	U+0080 ... U+07FF	128 ... 2047	2 байта	'Р' → 0xD0 0xA0
	греческий, кириллица, арабский			
	U+0800 ... U+FFFF	2048 ... 65535	3 байта	'我' → 0xE6 0x88 0x91
	иероглифы			
	U+10000 ... U+10FFFF	65536 ... 1114111	4 байта	'😄' → 0xF0 0x9F 0x98 0x80
	редкие символы, эмодзи			

В 2003 году стандарт UTF-8 был намеренно ограничен 4 байтами, чтобы его кодовое пространство совпадало с UTF-16 и было не больше U+10FFFF

UTF-8

как задаётся двухбайтный символ:

первый байт: **110.....**

второй байт: **10.....**

– то есть из 16 бит, только 11

отдаются на кодирование числа (5 + 6)

0x0080 ... 0x07FF

3-х-байтный:

первый байт: **1110.....**

второй байт: **10.....**

третий байт: **10.....**

– то есть из 24 бит, только 16

отдаются на кодирование числа

0x0800 ... 0xFFFF

4-х-байтный:

1-й байт: **11110...**

2-й байт: **10.....**

3-й байт: **10.....**

4-й байт: **10.....**

– то есть из 32 бит, только 21

отдаются на кодирование числа

0x10000 ... 0x10FFFF

смайлик относится к 4-х-байтному UTF-8

```
smb = '😄'
print(smb, ord(smb), hex(ord(smb)), bin(ord(smb)))
# 😄      128512      0x1f600      0b111110110000000000
```

эти биты номера символа нужно разместить
в шаблоне UTF-8 для 4-х байт начиная справа

такие задачи
порешаем на
практике

11110...	10.....	10.....	10.....
11110000	10011111	10011000	10000000
<u>F0</u>	<u>9F</u>	<u>98</u>	<u>80</u>

```
# URL-кодировка (percent-encoding)
from urllib.parse import quote
encoded = quote(smb)
# '%F0%9F%98%80'
```

```
utf8_bytes = smb.encode('utf-8')
print(utf8_bytes) # b'\xf0\x9f\x98\x80'
# эти байты реально хранятся в файле или передаются по сети
```

```
char = utf8_bytes.decode('utf-8') # декодируем в символ
print(char) # 😄 – проверяем получился ли снова смайлик
```

```
code_point = ord(char) # получаем Unicode символа
print(hex(code_point)) # 0x1f600
```

в интернете всё передаётся в виде байтов
текст, картинки, видео
перед отправкой преобразуются в байты

Строки

последовательности символов

работу со строками рассмотрим
в рамках практических занятий



Контрольные вопросы

Кодирование, шифрование, хеширование

Равномерное и неравномерное кодирования

Системы Бэкона, Бодо, Морзе

Принцип Фано

Ключевая идея для хранения символов

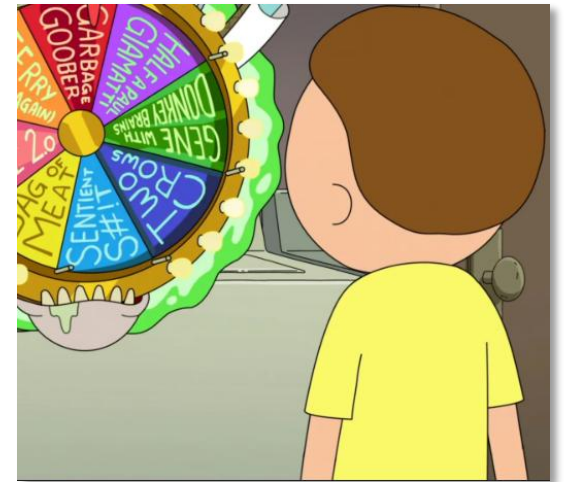
в памяти компьютера

Что такое кодировка

История развития кодировок

Стандарт UTF-8, особенности формата:

1 байт, 2 байта, 3 байта, 4 байта



Вопрос 2

Кодирование чисел

Лейбниц Готфрид Вильгельм (1646-1716) – первым стал применять с 1679 года двоичную систему для кодирования чисел и начал проектировать механическую вычислительную машину на основе двоичной системы

`gitverse.ru/band/TOI`

Кодирование чисел

Целые (точно)

1, 3, 16, ...

рассмотрим разрядную сетку бит размером N и беззнаковые целые:
диапазон $0 - 2^{N-1}$

сколько разных чисел
можно кодировать
такой разрядной
сеткой ?

Вещественные (приблизённо)

0.1, 0.125, 3.14159

пример: $0.1 + 0.2 \neq 0.3$

рассмотрим отдельно представление в двоичной форме для:

0.5 0.125 0.1 0.2

Кодирование чисел

Напишем функции для

представления в двоичной форме

- целых чисел

- вещественных чисел $0 \leq \text{num} < 1$:

0.5 0.125 0.1 0.2

на практическом занятии

рассмотрим причину погрешности

Хранение вещественных чисел

Python

float - число с плавающей точкой

- используется стандарт IEEE 754
- хранится в 64 битах (8 байтах)

1 бит — знак (0 — положительное, 1 — отрицательное)

11 бит — экспонента (порядок)

52 бита — мантисса (дробная часть)



Хранение чисел

длинная арифметика в Python

```
import sys
```

```
x, y, t, p = 12, 1.2, 2**24, 2**30
```

```
print(sys.getsizeof(x)) # 28  
print(sys.getsizeof(y)) # 24  
print(sys.getsizeof(t)) # 28  
print(sys.getsizeof(p)) # 32
```

- всё хранится как объект

- **float**:

 - 8 байт под число, 16 байт служебное

- **int**:

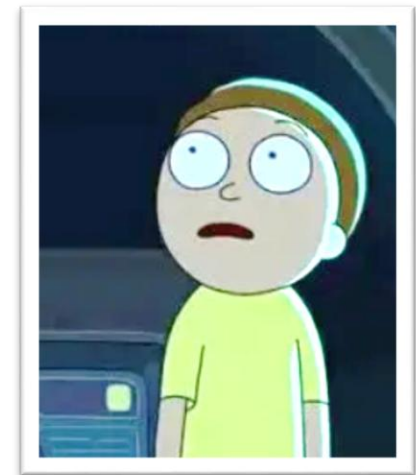
 - 4 байта под число, 16 - служебное,
- и 8 ссылка на след блок

- по мере увеличения значения int блоки по 4 байта добавляются
- Python поддерживает длинную арифметику (по умолчанию лимит 4300 цифр)
 - можно установить больше: `sys.set_int_max_str_digits(10_000)`
 - можно отключить ограничение: `sys.set_int_max_str_digits(0)`
- проверить: `big_num = int("1" * 5000)`
- ограничение - размер оперативной памяти

Контрольные вопросы

Формат длинной арифметики для целых чисел

Проблема точности вещественных чисел



Теоретические основы информатики

Раздел 1. Информация и данные

Тема 2: Кодирование информации

2026

Беляков Андрей Юрьевич